

SE116 PROJECT OBJECTVILLE – TEAM 07 – PROJECT REPORT

**1) Where did we use inheritance, polymorphism, abstract classes and interfaces?
(We need to state the reasons for using them in each part of the code.)**

-_*_-*_-*_-*_-*_-*_-*_-*_-*_-*_ [Answer to question 1] -_*_-*_-*_-*_-*_-*_-*_-*_-*_-*_

1. Abstract Classes

We used abstract classes as blueprints for the simulation objects. They hold the default behaviors and used for being sure no code is duplicated for no reason.

Cell (Base Class)

- **Where it's used:** It is the root of the entire map system. All the classes on the map extend Cell.
- **Reasons:**
 - **Behavior:** Every tile on a 2D map grid must have coordinates (x, y) and a symbol. Defining these properties and their getters/setters in Cell avoids duplicating code across a lot of classes.
 - **Grouping :** It declares abstract behaviors like isConnectable(). This helps the BFS algorithm, which utility providers use.
 - **Prevents user errors:** The user could mistakenly create a Cell class with nothing in it, Cell being abstract prevents this problem.

B. UtilityProvider

- **Where it's used:** Parent class for PowerPlant, WaterPumpingStation, and InternetHub.
- **Reasons:**
 - **Common attributes:** All utility providers produce resources and have a utilityType and capacity.
 - **BFS:** By default, all utility providers are connectable to the distribution network. UtilityProvider overrides isConnectable() to return true so that its subclasses need to implement this repeatedly.

C. Service

- **Where it's used:** Parent class for public services like Hospital, PoliceStation, and School.
- **Reasons:**
 - **Common Attributes:** Every service has an active radius and a serviceType.
 - **Grid Isolation:** Services do not have connectability for BFS. So, Service overrides isConnectable() to return false for all services.

D. Zone

- **Where it's used:** Parent class for generator cells (Housing, Commercial, Industrial).
 - **Reasons:**
 - **Resources :** All zones must keep track of local resources, active utility grids, coverage flags and incoming generated things (Population, Goods, Lifestyle).
 - **Ticks :** It provides the implementation of `resetTurnData()`, making sure all subclasses can clear their state at the start of a new tick.
 - **Different Behaviors:** While all zones consume utilities through the same mechanics (`consumeUtility()`), they have vastly different leveling mechanics. Zone defines `updateLevelAndOutput()` as an abstract method, forcing each zone type to define its custom leveling rules.
-

2. Interfaces

We used interfaces only once this project. We used it to give Zone type classes (Industrial, Housing ve Commercial) the ability to consume the resources with `UtilityConsumer`.

UtilityConsumer

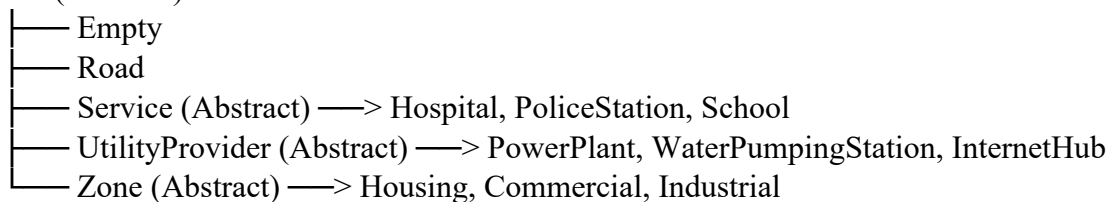
- **Reasons:**
 - **Separation of Capabilities:** Not every entity on the map consumes resources (for example Road or Empty tiles do not use water or electricity). Implementing `UtilityConsumer` allows us to assign consuming capabilities only to entities that require them.
 - **Flexibility:** If we later introduce a different building that requires power to operate, we cannot inherit from Zone because Java does not allot us to inherit from more than one instance. Instead, we can implement the `UtilityConsumer` interface. This way we can give the utility consuming ability to whatever class we want easily.
-

3. Inheritance

Inheritance is used to foster the code reusability.

- **System:**

Cell (Abstract)



- **Reasons:**
 - **Relationship:** This layout reflects as an is-a relationship.
 - **Polymorphic Arrays:** We made it so all the structural types are inheriting from Cell, so we can read the map and assign all the Symbols as a Cell in a Cell[][] array. This is the base of what we did in this project.
-

4. Polymorphism

Polymorphism allows our engine to treat different cells while executing behavior specific to their subtypes.

A. Subtype Polymorphism (Upcasting)

- **Where it's used:** Inside CityMap.java and SimulationManager.java during map rendering, BFS pathfinding, and service coverage mapping.
- **Reasons:**
 - When scanning neighbors in pathfinding (getNeighbors()), the engine queries a Cell pointer. It can move seamlessly from a Road object to a Housing object because both are treated as generic Cell elements. This simplifies the BFS algorithm by a lot.

B. Method Overriding

This is where polymorphism is used most in our project:

- **isConnectable() Overriding:**
 - Defined as abstract in Cell, overridden differently by Road (returns true), Empty (returns false), and Service (returns false).
 - During utility distribution in SimulationManager.java, the BFS pathfinder looks a class' isConnectable(). The engine does not care what subclass the neighbor is; it calls isConnectable(). This avoids unnecessary if-else or instanceof uses.
- **consumeUtility() Overriding:**
 - In the base class Cell, consumeUtility() is defined to do nothing and return the incomingAmount untouched:
- **updateLevelAndOutput() Overriding:**
 - In SimulationManager.java, when updating all zones, we invoke updateLevelAndOutput() on each zone object.

Through polymorphism, Java automatically triggers Housing.updateLevelAndOutput() for residential sectors, Commercial.updateLevelAndOutput() for commerce, and

Industrial.updateLevelAndOutput() for factories, each executing its respective leveling formulas without the caller knowing the actual class type.

2- Is our design open to extension for new types?

- *If a new type of zone is introduced, say, “governmental” which requires population and produces “law” which will be used to create level 4 for zones, which parts of your code must change?*
- *How about a new service cell type, such as “parks” which serves as “recreational” which parts of your code must change?*

-*--*_*_*_*_*_*_*_*_*_*_*_*_*_*_*_* [Answer to question 2] -*-**-*_*_*_*_*_*_*_*_*_*_*_*_*_*_*_***

- *Is our design open to extension for new types?*
 ⇒ I think our design is kind of open for new types but there is a couple of hard coded methods. For example, the switch case in the class assignment section and manual resource routing in simulationmanager. But it is not that hard to implement new zones because our object oriented system is upgrade-friendly in my opinion.

- *Implementation of new zone governmental.*

(i think governmental would be better and easier to implement as a Service but it's okay.)

- ➔ First, we need to create a new zone class.
`com.objectville.model.zones.Governmental.java`
- ➔ For the second step, we need to extend it with `Zone`, and define its constructor with a symbol, (G is not used yet so G is okay.)
- ➔ After that, we should implement `updateLevelAndOutput()` where it checks utility supply, consumes received population, and updates its level and output (generating law based on its level).
- ➔ Now, it is time for resource pool. We need to update it to have new variable `private int law`; getter, setters with methods `addLaw(int law)` and `consumeLaw(int law)`. We should also update `printPoolState()` to show law information as well.
- ➔ And we shouldn't forget the switch case in `generateCells()` method, we should also add a case for symbol 'G' that assigns governmental to the G cell.
- ➔ We should update `getTotalUtilityDemand()` to include governmental as well.
- ➔ In `Zone` Class, we should do the following:
 - add `public int receivedLaw = 0`;
 - add `receivedLaw = 0` in `resetTurnData()`
- ➔ Also we should update the other zones (Housing, Industrial, Commercial) to have the ability to upgrade to level 4 with `receivedLaw > 0` along with other conditions.
- ➔ Finally, we should update the simulation. We should include `Governmental` in `distributeResources`, count how many G's there are. Add `Governmental` to `totalConsumers` and make governmental to receive a part of population. Calculate how many law is distributed and assign to the zones. Finally, we should reset the

- *Implementation of new Service Parks*

- ### 3- State where in your code you have used memory-resident structures or generic types.

1. In com.objectville.model.zones.Zone

- `Map<String, Integer> currentUtilities`: A generic Map mapping String keys to Integer values, initialized as a memory-resident `HashMap`. It stores the levels of utilities (Electricity, Water, Internet) consumed by each zone.
- `Map<String, Boolean> currentServices`: A generic Map mapping String keys to Boolean values, initialized as a memory-resident `HashMap`. It tracks whether different services (Security, Health, Education) are available to the zone.

- Queue<Cell> queue: A generic Queue holding Cell objects, implemented using a memory-resident LinkedList◊. It serves as the frontier queue for utility distribution.

- ### 3. In `com.objectville.engine.CityMap`

- `ArrayList<String> lines` (in `loadMap`): A generic `ArrayList` holding `String` objects, used to store the lines read from the map file into memory.
- `List<Cell> neighbors` (in `getNeighbors`): A generic `List` holding `Cell` objects, implemented with `ArrayList`, used to dynamically aggregate and return adjacent cells.
- `char[][] charGrid` & `Cell[][] cellGrid`: Memory-resident 2D arrays storing the loaded characters and instantiated game objects representing the current game map.

-*--*_*-_-*_--*_--*_--*_-- [Answer to question 4] -*-**-*_*-_-*_--*_--*_--*_--*_--**

1. Packaged Units (Java Packages) Our codebase is using Java packages to group related classes, making sure we know what we are working with and this way we can split the work with our group members.

- **Simulation Engine:** The core logic of the game, including classes like CityMap, SimulationManager, and ResourcePool, is packaged under com.objectville.engine.
- **Domain Models:** The grid elements and structures are stored in com.objectville.model. This is further divided into specific packages to separate different Classes.
 - com.objectville.model.zones groups zone classes like Housing, Commercial, and Industrial.
 - com.objectville.model.providers groups utilityproviders like PowerPlant, WaterPumpingStation, and InternetHub.
 - com.objectville.model.services is dedicated to city services.
- **Interfaces:** UtilityConsumer interface is stored in com.objectville.interfaces.
- **Exceptions:** Custom error-handling classes (ConfigurationException and InvalidMapException) are stored in com.objectville.exception.

Based on the structure of the classes, lambda expressions are not present in the classes or basic constructors.

(Maybe there was places where lambda expressions would be better to use in some cases, but since we aren't really used to using lambdas, we didn't use any.)

5- Describe how your code handles the map files to create the map in your program.

-_*_-*_-*_-*_-*_-*_-*_-*_-*_-*_ [Answer to question 5] -_*_-*_-*_-*_-*_-*_-*_-*_-*_-*_-

In summary, our project uses `BufferedReader` to read the text file provided in args while executing the program and counts the row and column count, separates all the symbols in the map to assign the Classes with a switch case.

1. Starting

The program's entry point, `ObjectVilleGame`, receives the map file path via `args`. If the required map file or tick number arguments are missing or incorrect, system throws a `ConfigurationException`.

2. File Reading

The CityMap class is responsible for the actual loading and reading of the map file. It utilizes standard Java IO components like File, FileReader, and BufferedReader to access and read the file contents.

3. Assigning

As CityMap reads the text file, it assigns symbols to their Classes with a switch case. Every element on the map is a subclass of the abstract Cell class.

4. Memory Storage

As these Cells are instantiated, CityMap organizes and stores them using dynamic, memory-resident data structures like `java.util.ArrayList` and `java.util.List` to construct the active simulation grid in memory. (charGrid, cellGrid)

5. Error Handling

If the parser encounters structural issues, invalid characters, or logical violations within the map file during this process, it uses our custom `InvalidMapException` to handle the errors safely and standard file-handling exceptions like `FileNotFoundException` and `IOException` are used during the I/O operations.

6- Write a postmortem section. A postmortem discusses what went right and what went wrong in your project. Did you come across any common pitfalls? If so, how did you recover? If not, what did you do to not fall into these pitfalls?

**_*_*_*_*_*_*_*_*_*_*_*_*_*_ [Postmortem] _*_*_*_*_*_*_*_*_*_*_*_*_*_*_*_

-> Firstly, it was hard for our team to get contacts. Communication was a big problem since we didn't know each other before. After getting the phone numbers of each other, we planned to meet up to talk about the project but couldn't meet altogether because of the differences in our daily plans and course schedules. After making the github repository, it was hard to start

Github problems on the other hand is a whole different thing, 3 of us were doing our first group project in github and learning how to use it, file overlapping and sometimes new changes were not committing were a problem. But the more we used github, we adapted.

Arda Temel Birinci